Build Your First MCP Server in 15 Minutes (Complete Code)

The three building blocks that make LLM-tool integration surprisingly simple

Paolo Perrone

Following

6 min read

.

16 hours ago

Listen
Share
More

Six months ago, I spent two weeks building a “smart” customer support agent. It could answer questions, look up order status, and even process refunds. I was proud of it.

The integration code was a nightmare. Custom API calls everywhere. JSON parsing that broke if a field was missing. A 400-line function just to handle tool routing. But it worked — mostly.

Then I demoed it to the team.

“What’s the status of order 12345?” someone asked.

The agent confidently replied with the order details. Great. Then someone asked a follow-up question, and the agent tried to call the order lookup

function again. Except this time, my brittle parsing logic choked on an edge case. The whole thing crashed. In front of everyone.

I spent that night debugging, and realized the problem wasn't my logic. It was the architecture. I'd built a house of cards trying to connect an LLM to external tools using custom code. Every new tool meant more custom parsing. Every edge case meant more if-statements.

That failure led me to MCP.

The Model Context Protocol is an open standard that does one thing well: it gives LLMs a clean, consistent way to discover and use external tools. No more custom parsing. No more brittle integrations. Just a protocol that works.

Once I understood MCP, I rebuilt that same agent in an afternoon. And it hasn't crashed since.

I Hate When People Overcomplicate This Stuff. Most MCP explanations start with JSON-RPC specifications and transport layer discussions. That's backwards.

You don't need to understand the protocol internals to use it — just like you don't need to understand HTTP to build a web app.

Here's what you actually need: three concepts and about 15 minutes.

The Three Building Blocks

MCP has three building blocks. That's it.

1. **Server** — The thing that exposes your tools. It's a Python script that says "here are the functions an LLM can call." You run it, and it listens for requests.

2. **Tool** — A function you want the LLM to use. Could be anything: fetch weather, query a database, send an email. You write it like a normal Python function, add a decorator, and MCP handles the rest.

3. **Client** — The thing that connects to your server and calls tools. In production, this is usually your LLM application. For testing, FastMCP gives you a client that works out of the box.

Server exposes tools. Client calls tools. That's the entire mental model.

Everything else — transports, JSON-RPC, capability negotiation — is implementation detail. You don't need to think about it until you're scaling to production.

Let's build one.

Step 1: Install FastMCP

FastMCP is the Python framework that makes MCP simple. One install, no configuration.

```
pip install fastmcp
```

That's it. No virtual environments required for this tutorial (though you'd want one in production).

Step 2: Create Your Server

Create a file called `my_server.py`:

```
from fastmcp import FastMCP

# Initialize the server with a name
mcp = FastMCP("my-first-server")

# Define a tool using the @mcp.tool decorator
```

```python
@mcp.tool
def get_weather(city: str) -> dict:
    """Get the current weather for a city."""
    # In production, you'd call a real weather API
    # For now, we'll return mock data
    weather_data = {
        "new york": {"temp": 72, "condition": "sunny"},
        "london": {"temp": 59, "condition": "cloudy"},
        "tokyo": {"temp": 68, "condition": "rainy"},
    }

    city_lower = city.lower()
    if city_lower in weather_data:
        return {"city": city, **weather_data[city_lower]}
    else:
        return {"city": city, "temp": 70, "condition": "unknown"}

# Run the server
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

Let's break down what's happening:

- `FastMCP("my-first-server")` creates your server with a name
- `@mcp.tool` is the decorator that turns any function into an MCP tool
- The docstring becomes the tool's description (LLMs use this to understand when to call it)
- Type hints (`city: str`, `-> dict`) tell MCP the expected inputs and outputs
- `transport="stdio"` means the server communicates via standard input/output (perfect for local testing)

That's your entire server. 15 lines of actual code.

Step 3: Create a Client to Test It

Create a file called `test_client.py`:

```python
import asyncio
from fastmcp import Client

async def main():
    # Point the client at your server file
    client = Client("my_server.py")

    # Connect to the server
    async with client:
        # List available tools
        tools = await client.list_tools()
        print("Available tools:")
        for tool in tools:
            print(f"  - {tool.name}: {tool.description}")

        print("\n" + "="*50 + "\n")

        # Call the weather tool
        result = await client.call_tool(
            "get_weather",
            {"city": "Tokyo"}
        )
        print(f"Weather result: {result}")

if __name__ == "__main__":
    asyncio.run(main())
```

Key points:

- `Client("my_server.py")` tells the client which server to connect to
- `async with client:` handles the connection lifecycle automatically
- `list_tools()` discovers what tools are available (this is MCP's dynamic discovery)
- `call_tool("get_weather", {"city": "Tokyo"})` invokes the tool with parameters

Step 4: Run It

Open your terminal and run:

```
python test_client.py
```

You should see:

```
Available tools:
  - get_weather: Get the current weather for a city.
```

```
==================================================Weather
result: {'city': 'Tokyo', 'temp': 68, 'condition': 'rainy'}
```

That's it. You just built an MCP server and called it from a client.

Step 5: Add More Tools

The power of MCP is how easy it is to add capabilities. Let's add two more tools to our server:

```
from fastmcp import FastMCP
from datetime import datetime

mcp = FastMCP("my-first-server")

@mcp.tool
def get_weather(city: str) -> dict:
    """Get the current weather for a city."""
    weather_data = {
        "new york": {"temp": 72, "condition": "sunny"},
        "london": {"temp": 59, "condition": "cloudy"},
        "tokyo": {"temp": 68, "condition": "rainy"},
    }
    city_lower = city.lower()
    if city_lower in weather_data:
        return {"city": city, **weather_data[city_lower]}
    return {"city": city, "temp": 70, "condition": "unknown"}

@mcp.tool
def get_time(timezone: str = "UTC") -> str:
    """Get the current time in a specified timezone."""
    # Simplified - in production use pytz or zoneinfo
    return f"Current time ({timezone}):
{datetime.now().strftime('%H:%M:%S')}"

@mcp.tool
def calculate(expression: str) -> dict:
    """Safely evaluate a mathematical expression."""
    try:
```

```
        # Only allow safe math operations
        allowed_chars = set("0123456789+-*/.() ")
        if not all(c in allowed_chars for c in expression):
            return {"error": "Invalid characters in expression"}

        result = eval(expression)  # Safe because we validated input
        return {"expression": expression, "result": result}
    except Exception as e:
        return {"error": str(e)}
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

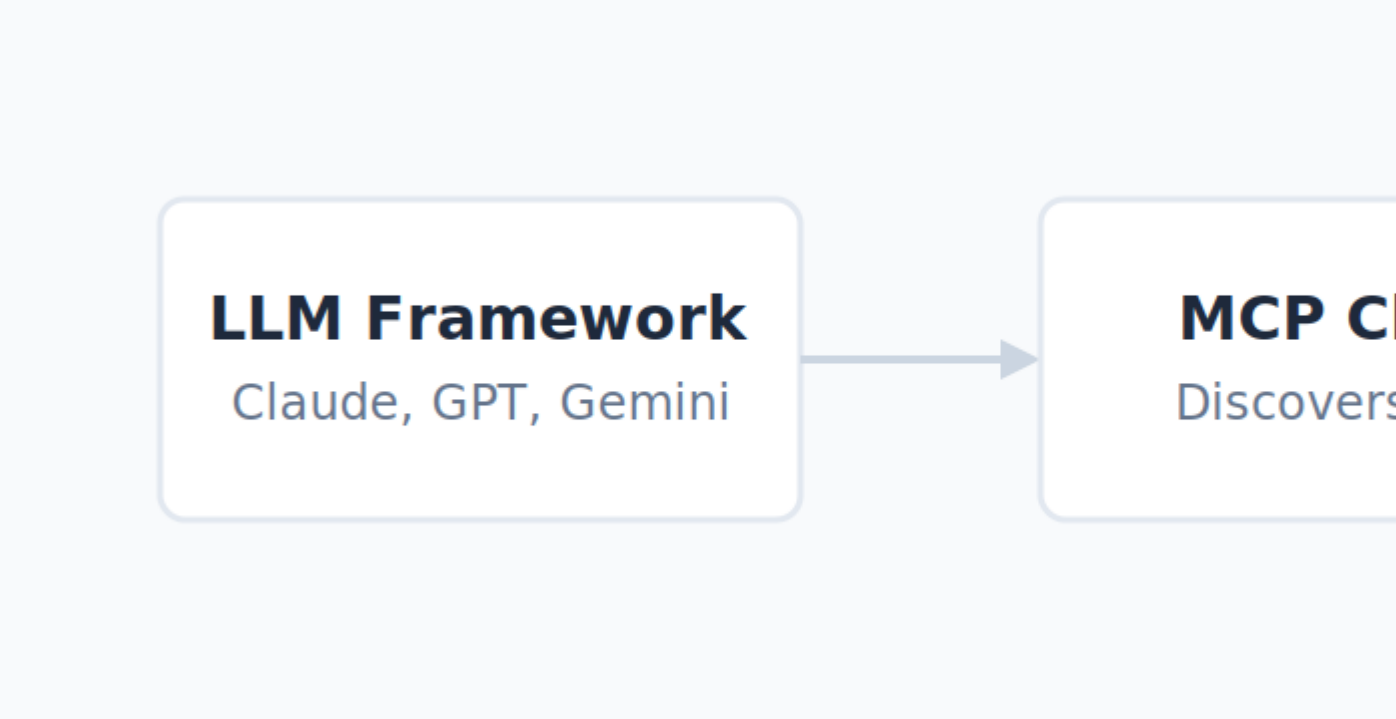
Run your test client again — it will automatically discover all three tools:

```
Available tools:
  - get_weather: Get the current weather for a city.
  - get_time: Get the current time in a specified timezone.
  - calculate: Safely evaluate a mathematical expression.
```

No configuration changes. No routing logic. You added tools, and MCP made them available.

What's Next: Connecting to an LLM

The client we built is for testing. In production, your LLM framework connects as the client. Here's how that looks conceptually:



How MCP connects your LLM to external tools: the framework calls the client, which discovers and invokes tools from your server.

The server code you wrote doesn't change. That's the point of MCP — you build tools once, and any MCP-compatible client can use them.

For production deployments, you'd also switch from `stdio` transport to `http`:

```
if __name__ == "__main__":
    mcp.run(transport="http", host="0.0.0.0", port=8000)
```

This exposes your MCP server as an HTTP endpoint that remote clients can connect to.

The Real Lesson

Remember that customer support agent that crashed during my demo?

The problem wasn't that I couldn't write working code. I could. The problem was that I was solving the wrong problem. I was building custom integrations when what I needed was a standard protocol.

MCP isn’t magic. It’s plumbing. Good plumbing.

It handles the boring stuff — discovery, routing, serialization — so you can focus on what matters: the tools themselves. Your weather function, your database query, your email sender.

The real insight?

Good abstractions make hard problems disappear.

You now have a working MCP server. In 15 minutes. The next step is to connect it to your actual LLM and build something useful.

I’d start with whatever tool you wished your agent had last week.

Artificial Intelligence

Machine Learning

Programming